MSML640 Mid-Term Project 1

Classical CV for Traffic Sign Detection

March 30, 2026

External sources and tools

We used the following external sources and tools while working on this project:

- Tiny LISA Traffic Sign Detection Dataset – Kaggle: dataset link
- CLAHE to improve local contrast in images – GeeksforGeeks: article
- Using HSV color segmentation idea – ChatGPT: "My SIFT algorithm is not working well for some images even though we did preprocessing like upscaling the image and applying sharpening. What else can be implemented to improve the performance? (I am using real-world traffic board sign images.)"
- `HSV_COLOR_RANGES` – ColorPicker: colorpicker.me
- Bitwise AND, OR on images – PyImageSearch: blog post
- Pickle to cache the dataset – GeeksforGeeks: article
- `cv2.KeyPoint` (SIFT keypoints) – Tessl.io OpenCV Python documentation: documentation
- Adding text to images – GeeksforGeeks: article
- Adding rectangle to image – OpenCV official documentation: documentation

How to Run the Code

```
git clone git@github.com:suryajk-711/Computer-Vision-Project_1.git
```

1. Run the code
   (a) Custom dataset (captured using camera)
   ```
   python3 src/main.py dataset --annotations dataset/annotations.csv
   ```
   (b) Tiny LISA Dataset
   ```
   python3 src/main.py db_lisa_tiny --annotations db_lisa_tiny/annotations.csv
   ```
2. Run the Frontend code
   (a) cd src
   (b) python3 app.py
   (c) Go to `http://127.0.0.1:5000` and upload the image

Introduction

This project explores how well classical computer vision techniques can detect and classify traffic signs in realistic, messy conditions outside the lab. The focus is on three common road signs: STOP, Pedestrian Crossing and Signal Ahead for which we use real life data and the other classes use data from the LISA dataset. Rather than using modern deep learning models, the system relies entirely on traditional methods: HSV color segmentation to isolate sign-colored regions (such as red for STOP and yellow for warnings) and SIFT-based template matching to recognize which sign is present by comparing to a small set of hand-crafted template images. The main objective is to test this classical pipeline on real-world images and understand where it works, where it fails and why especially under challenging conditions like poor lighting, motion blur and cluttered backgrounds.

Datasets Used

Two datasets were used to evaluate the pipeline.

Self-Captured Real-World Images (around 70 images)

- Collected using a phone camera on local streets and intersections.
- Focused on three target classes: **STOP**, **Pedestrian Crossing** and **Signal Ahead**.
- Images are completely "in the wild", with:
  - Varying lighting (bright sun, deep shadows, overcast, dusk).
  - Different viewing angles (frontal, oblique, tilted).
  - Varying distances (close-up to small, distant signs).
  - Environmental noise such as cluttered backgrounds, partial occlusions (trees, poles, other cars), and motion blur.

Tiny LISA Traffic Sign Detection Dataset (900 samples, 9 classes)

- Public dataset (from Kaggle) containing traffic sign images and annotations.
- Includes 9 different sign classes, which extend beyond the three primary signs in the self-captured data.
- Provides more controlled and diverse examples for each class, often with clearer views and more consistent framing than the self-captured set.

What are Templates?

Templates are reference images of known traffic signs, organized into folders by class name (e.g., `templates/stop/`, `templates/yield/`). The pipeline supports nine classes: `keepRight`, `merge`, `pedestrianCrossing`, `signalAhead`, `speedLimit25`, `speedLimit35`, `stop`, `yield`, and `yieldAhead`. When a query image comes in, it is compared against these known templates to find the closest match.

How Many Images are there per Class?

Five template images are used per class. SIFT descriptors are computed for each template, and during matching only the top three scoring templates (`TOP_K_TEMPLATES = 3`) are averaged to produce the final class score.

Why Templates are Cached

Computing SIFT descriptors for all templates on every run is computationally expensive. After the first run, the descriptor store (keypoints and descriptors for all templates) is serialized and saved as a `.pkl` pickle file (`templates_cache.pkl`). On subsequent runs, this cache is loaded directly, avoiding recomputation and significantly reducing startup time.

Pipeline

Query Image Pipeline: BGR → Color Mask (class-specific) → Upscale if small → Grayscale → CLAHE → Sharpening.

Upscaling

If the image is smaller than 64×64 pixels, it is upscaled using bicubic interpolation (`INTER_CUBIC`) so that SIFT has enough pixels to detect meaningful keypoints.

Grayscale Conversion

SIFT operates on single-channel images, so the color image is converted to grayscale before feature extraction.

CLAHE

Contrast Limited Adaptive Histogram Equalization (CLAHE) enhances local contrast, helping the detector handle shadows, glare, and uneven lighting conditions common in real-world road images.

Sharpening

An unsharp mask is applied by subtracting a blurred version of the image from the original. This enhances edges, making sign borders and internal features more distinct and producing stronger, more repeatable SIFT keypoints.

Template Image Pipeline: BGR → Color Mask → Upscale → Grayscale.

Templates are clean, studio-quality images, so CLAHE and sharpening are skipped. This avoids over-enhancing already clean images and keeps template descriptors stable over time.

HSV Color Segmentation

Each class is associated with specific colors (e.g., `stop` → red + white, `merge` → black + yellow). Before computing SIFT descriptors, a binary mask is built in HSV color space for those specific colors, and pixels outside those colors are zeroed out.

Without color segmentation, SIFT would extract keypoints from irrelevant background regions (trees, road surface, sky) introducing noise into descriptor matching and causing frequent misclassifications. By masking to only the colors that belong to a given class, the descriptors become class specific and matching becomes significantly more discriminative which noticeably improves classification accuracy.

Region oF Interest

When an `annotations.csv` file is provided, each row contains the filename along with bounding box coordinates (x_1, y_1, x_2, y_2) that mark the exact location of the sign in the image. The pipeline crops the image to that bounding box and resizes the crop to 128×128 before running SIFT.

Without an ROI, the pipeline runs SIFT on the entire image, where the sign may occupy only a small fraction of the pixels. Most keypoints are then extracted from irrelevant background, making matching unreliable. With the annotation-based ROI, the classifier receives a tightly cropped sign region, which greatly increases the density of relevant keypoints and improves prediction accuracy.

How the Final Class is Decided

For each of the nine classes independently:

1. The query image is masked with that class's color mask.
2. SIFT descriptors are computed on the masked query.
3. The query descriptors are matched against every template of that class using a BFMatcher with L2 norm and cross-check enabled.
4. Matches with distance above 250 are discarded as junk. If fewer than 8 good matches remain, that template is skipped.
5. A normalized score is computed as

$$\text{score} = \frac{\text{avg_distance}}{\text{coverage} + \varepsilon}$$

   where coverage is the fraction of query descriptors that found a good match. Lower score means a better match.
6. The top three template scores for that class are averaged to produce the final class score.

After all nine classes are scored, the class with the **lowest average score** is selected as the predicted label.

Overall pipeline of an uploaded image

Suppose a stop sign image is uploaded:

1. **Template descriptors** are loaded from the pickle cache.
2. `run_pipeline()` is called with the image path.
3. If annotations exist, the image is cropped to the bounding box and resized to 128×128; otherwise the full image is used.

4. `score_all_classes()` loops over all nine classes. For each class, it applies that class's color mask to the query, computes SIFT descriptors, and calls `score_single_class()`.
5. For the `stop` class, the red+white mask isolates the sign's relevant pixels. Good keypoint matches are found against the five STOP templates, yielding a low normalized score.
6. For unrelated classes such as `merge` (black+yellow mask), the query has almost no pixels passing the mask, so SIFT finds too few keypoints and the class is skipped.
7. `predict_class()` sorts all class scores in ascending order and returns the class with the lowest score, which in this case is `stop`.
8. The result is annotated and saved to the output folder under `outputs/stop/`.

Results

In this section we present qualitative results of the traffic sign detection system on a selection of test images. Each figure output image illustrating a particular success or failure case of the classical computer vision pipeline.

Quantitative Results

Tiny LISA Dataset

```
[4/4] Results:

==================================================
  SUMMARY
==================================================
  Total images   : 900
  Matched        : 236
  No match       : 664
  Load errors    : 0
==================================================


==================================================
  PREDICTION REPORT
==================================================
  Total annotated  : 900
  Correct          : 79
  Mispredicted     : 157
  Couldn't predict : 664
==================================================
```

Custom Dataset

```
[4/4] Results:
```

```
==================================================
  SUMMARY
==================================================
  Total images   : 70
  Matched        : 69
  No match       : 1
  Load errors    : 0
==================================================


==================================================
  PREDICTION REPORT
==================================================
  Total annotated  : 70
  Correct          : 12
  Mispredicted     : 57
  Couldn't predict : 1
==================================================
```

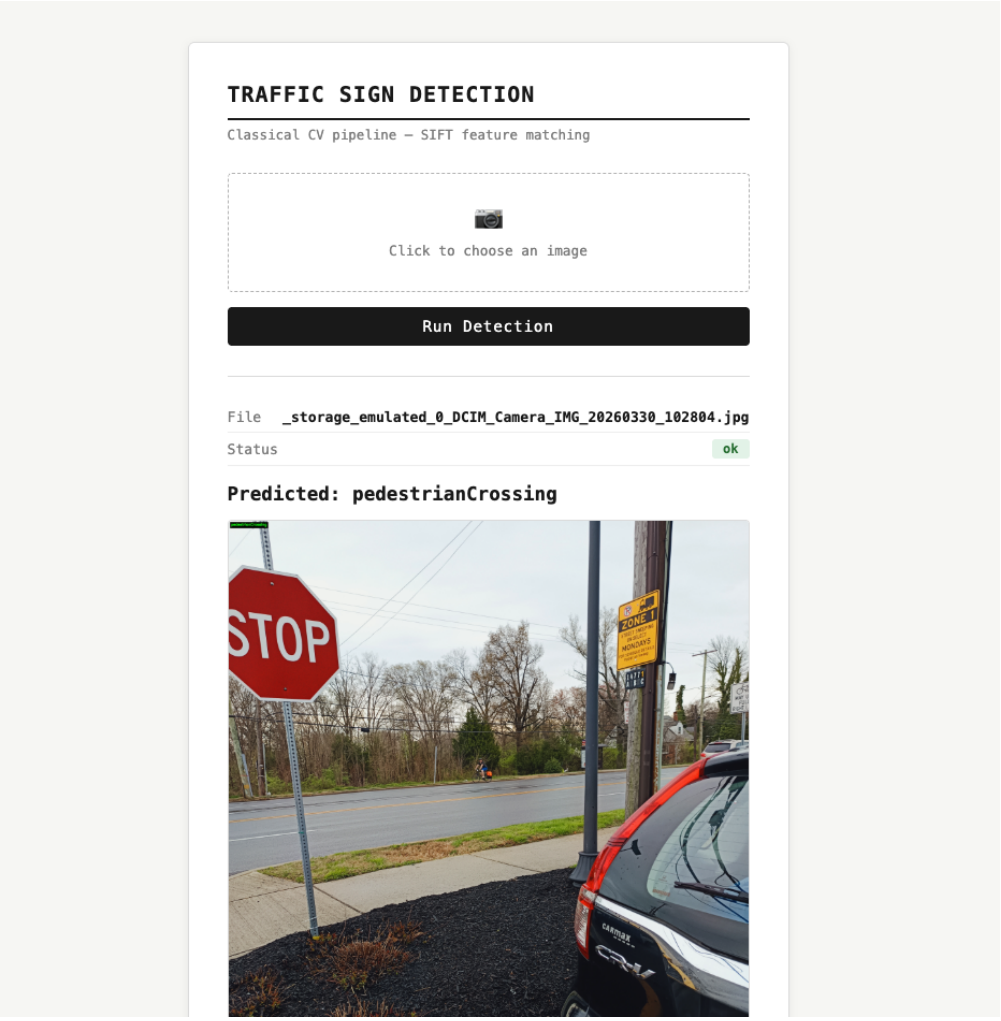

Figure 1: Successful detection of a STOP sign under good lighting and frontal view.

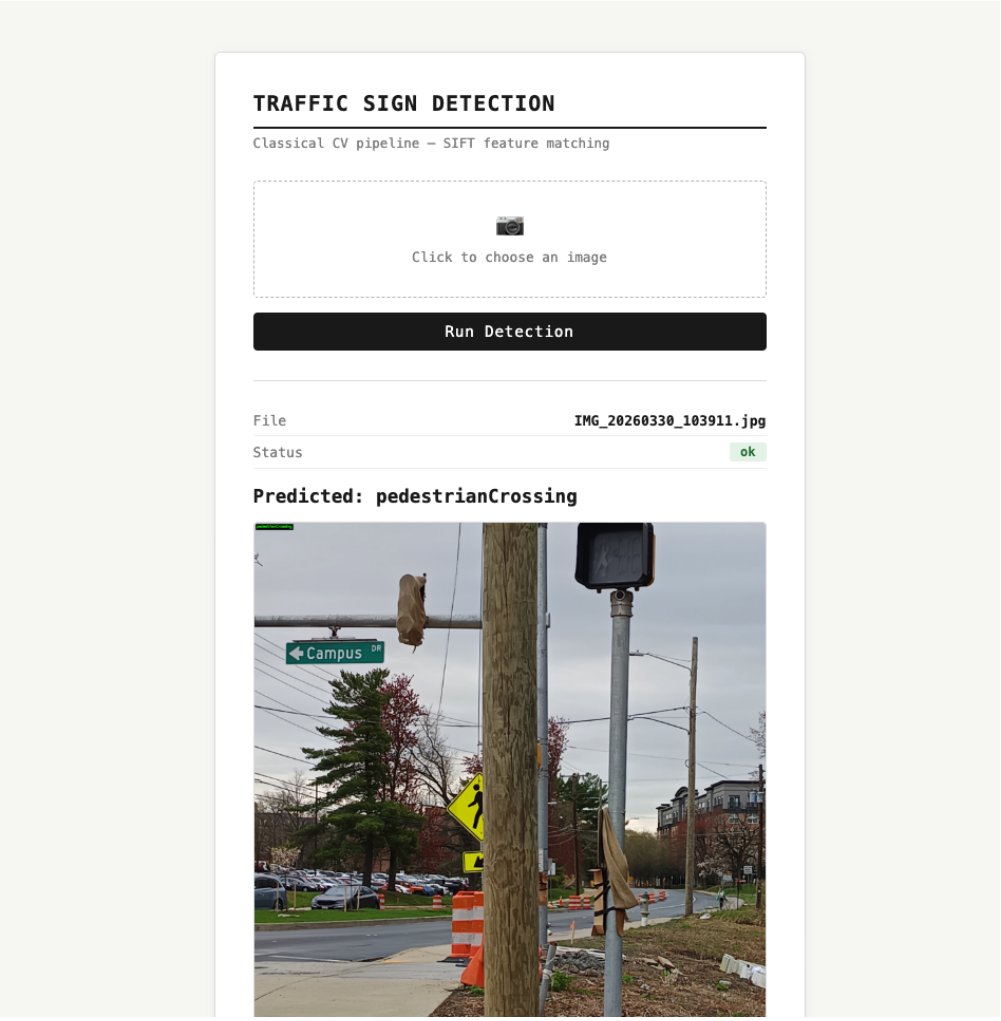

Figure 2: Pedestrian Crossing sign correctly classified at an oblique viewing angle.

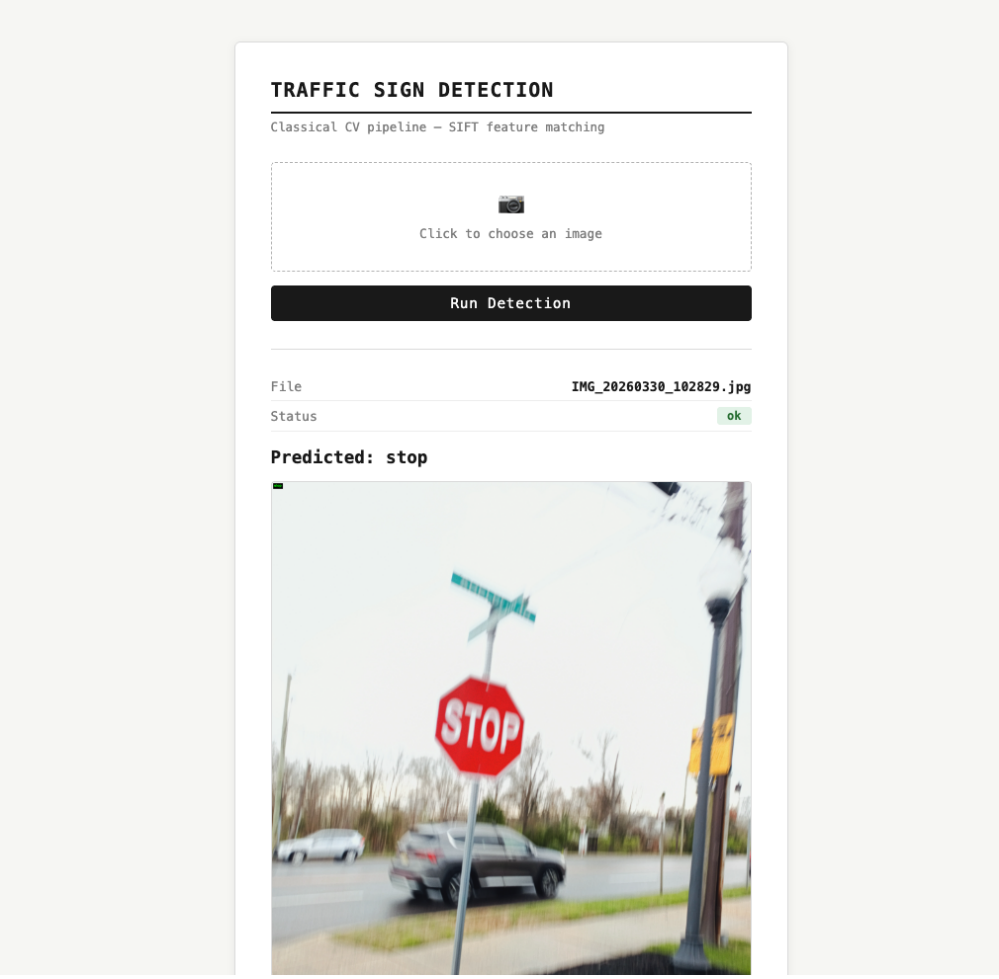

Figure 3: Successful detection of a STOP sign under blurred view in close up

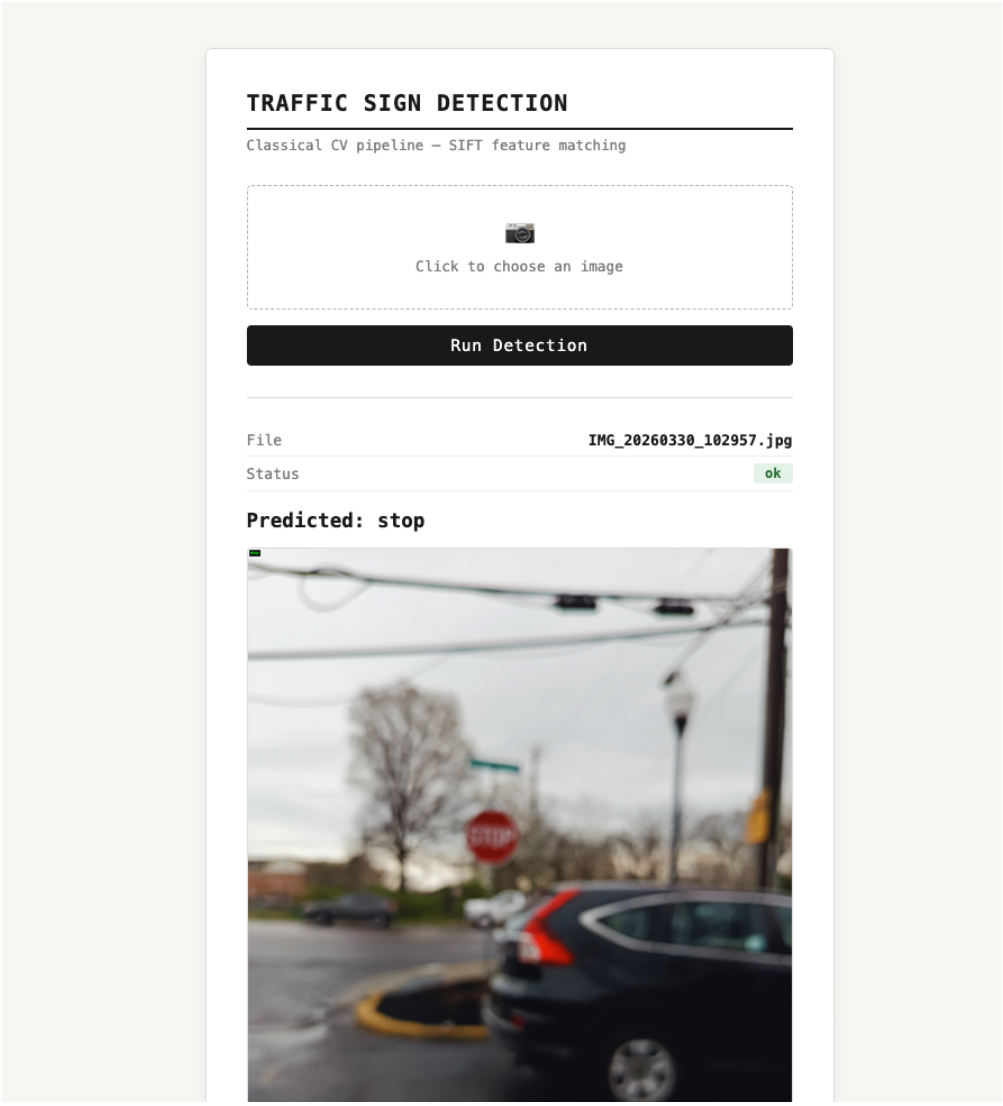

Figure 4: Successful detection of a STOP sign under blurred view from far

Things We Tried

Classical SIFT Matching Without Any Preprocessing

The first approach was straightforward compute SIFT descriptors directly on the raw grayscale query image and match them against template descriptors using BFMatcher. While SIFT itself is a robust feature detector, applying it to the full unprocessed image caused it to extract keypoints from everywhere like road surface, background vegetation, sky and other irrelevant regions. Since the sign itself may occupy only a small portion of the image, the majority of descriptors had nothing to do with the sign. This made the matching noisy and unreliable and the classifier frequently predicted the wrong class or failed to find any match at all. There was no mechanism to focus the descriptor extraction on the region or colors that actually mattered.

Sliding Window Detection

To handle images where no bounding box annotation was available, a sliding window approach was implemented. The image was divided into fixed 64×64 pixel windows and each crop was independently scored against all template classes. The intention was to localize the sign automatically without needing annotations.

However, this had two significant problems. First, the runtime increased dramatically scoring every window against all 9 classes with full SIFT matching meant hundreds of descriptor computations per image making the pipeline far too slow for practical use. Second, traffic signs are not always contained within a 64×64 region at a fixed grid position. Signs that fall between window boundaries get split across multiple crops, meaning no single window ever captures the complete sign. A partial sign view produces weak or misleading keypoints, and the classifier either fails to match or predicts the wrong class. The sliding window approach was ultimately abandoned in favor of annotation based ROI cropping.

Tuning HSV Color Ranges

Since color segmentation was identified as a key component, significant effort was spent tuning the HSV range boundaries for each color (red, white, yellow, black, green). The goal was to make the masks more precise so that only the most relevant pixels survived into SIFT. However, HSV thresholds are inherently sensitive to lighting conditions, camera exposure and image compression artifacts. A range that worked well under one lighting condition would over mask or under mask signs captured under different conditions. Widening the range brought in too much background noise and narrowing it removed too much of the sign itself. Despite multiple iterations, no single set of HSV ranges generalized well across all images in the dataset and the performance gains from tuning were marginal.

Future Work

One promising direction would be incorporating sign shape detection alongside the existing HSV color segmentation and SIFT matching. Traffic signs follow standardized geometric shapes like stop signs are octagons, yield signs are inverted triangles, speed limit signs are rectangles and detecting these shapes using contour analysis or Hough transforms could provide an additional discriminative signal before SIFT is even applied. This would help narrow down candidate classes based on shape alone, reducing the burden on descriptor matching.

However, this approach would still have a fundamental limitation. If the captured image contains only a partial view of the sign due to occlusion, the sign being at the edge of the frame or the vehicle passing the sign mid capture and the shape would be incomplete and unrecognizable. Shape based methods rely on the full geometric structure of the sign being visible and a half visible octagon or a clipped triangle provides no reliable shape information. Therefore, while shape detection could improve performance under ideal conditions, it would not resolve the core challenge of handling partially visible or occluded signs, which would ultimately require a learning-based detection approach to address robustly.